并行算法

主讲人：段贵多
Email:duanguiduo@163.com

第二章 并行算法概述

1. 并行算法分类
2. 并行算法表达
3. 并行算法的设计方法
4. 并行算法度量

并行算法的目标

- 并行算法的目标
- 由串行的“深而窄”变为“浅而深”
- 并行算法尽可能减少时间复杂性
- 通常通过增加空间复杂性

❖并行算法的定义

- 算法
- 并行算法：一些可同时执行的诸进程的集合，这些进程互相作用和协调动作从而达到给定问题的求解。

❖并行算法的分类

- 数值计算和非数值计算
- 同步算法和异步算法
- 分布算法
- 确定算法和随机算法

并行算法的分类

- 数值计算：基于代数关系运算的一类
- 非数值计算：基于比较关系运算的一类

- 同步算法：算法的多个进程的执行必须相互等待
- 异步算法：算法的多个进程的执行不需要相互等待
- 分布算法：由通信链路连接的多个节点，协同完成问题求解的一类并行算法

- 确定算法：算法的每一步都能明确地指明下一步应该如何行进的一类算法
- 随机算法：算法的每一步随机地从指定范围内选取若干参数，由其来确定算法下一步的走向

并行算法的特点

- 对并行处理机的使用效率有直接影响
  - **100%**的向量计算，速度达**1**亿次/秒
  - **75%**的向量计算，速度达**1400**万次/秒
  - **10%**的向量计算，速度达**400**万次/秒
- 对并行计算机有较强的依赖性
  - 处理机的连接方式
  - 存储器是否共享
  - 非共享存储器之间的连接
  - 存储器与处理机之间的连接

并行算法的表达

❖ 描述语言

- 可以使用类Algol、类Pascal等；
- 在描述语言中引入并行语句。

❖ 并行语句示例

- Par-do语句

```
for i=1 to n par-do
    ……
end for
```

- for all语句

```
for all Pi, where 0≤i≤k
    ……
end for
```

并行算法的同步

- 同步概念
  - 同步是在时间上强使各执行进程在某一点必须互相等待;
  - 可用软件、硬件和固件的办法来实现。
- 同步语句示例
  - **算法2.1 共享存储多处理器上求和算法**

    输入： $A=(a_0,\cdots,a_{n-1})$, **处理器数**p

    输出： $S=\sum a_i$

```
Begin
    (1)S=0
    (2)for all Pi where 0≤i≤p-1 do
        (2.1) L=0
        (2.2) for j=i to n step p do
                L=L+a_j
              end for
        end for
    (2.3) lock(S)
          S=S+L
    (2.4) unlock(S)
    end for
End
```

并行算法的通讯

- 通讯
  - 共享存储多处理器使用：global read(X,Y)和global write(X,Y)
  - 分布存储多计算机使用：send(X,i)和receive(Y,j)
- 通讯语句示例
  - 算法2.2 分布存储多计算机上矩阵向量乘算法

    输入：处理器数p, A划分为B=A[1..n,(i-1)r+1..ir],

    x划分为w=w[(i-1)r+1;ir]

    输出：P_1保存乘积AX

    Begin

    (1) Compute z=Bw

    (2) if i=1 then y_i=0 else receive(y,left) endif

    (3) y=y+z

    (4) send(y,right)

    (5) if i=1 then receive(y,left)

    End

并行算法设计方法

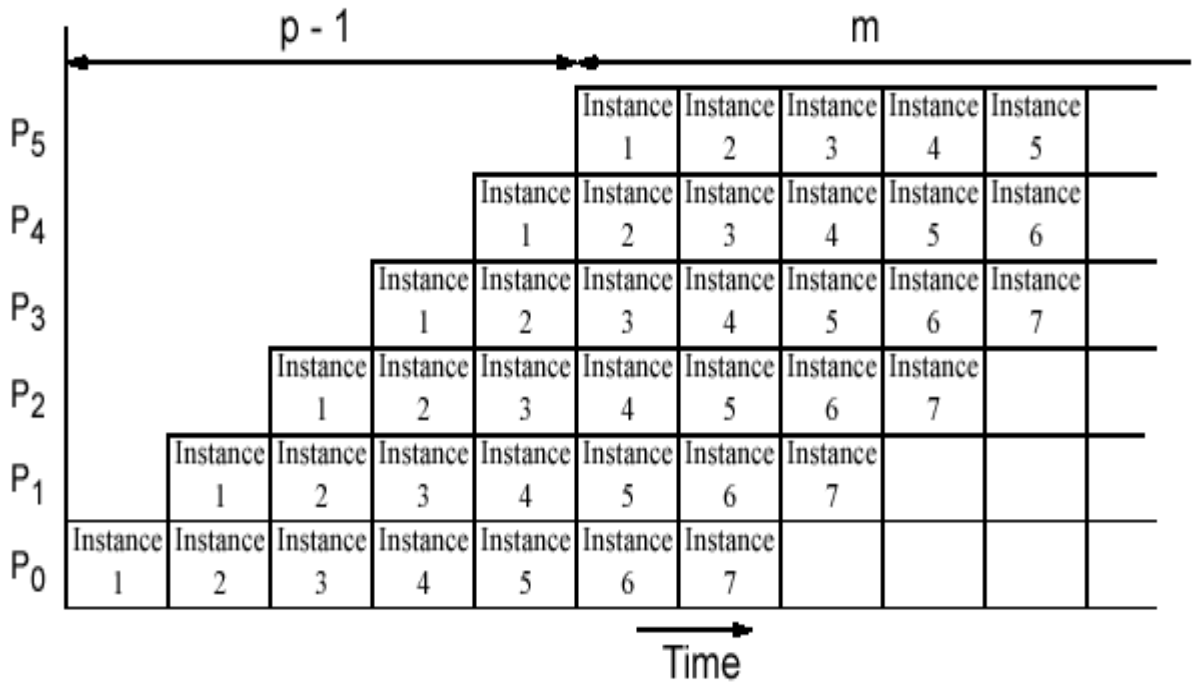
- 1. 流水线技术

❖ 设计思想

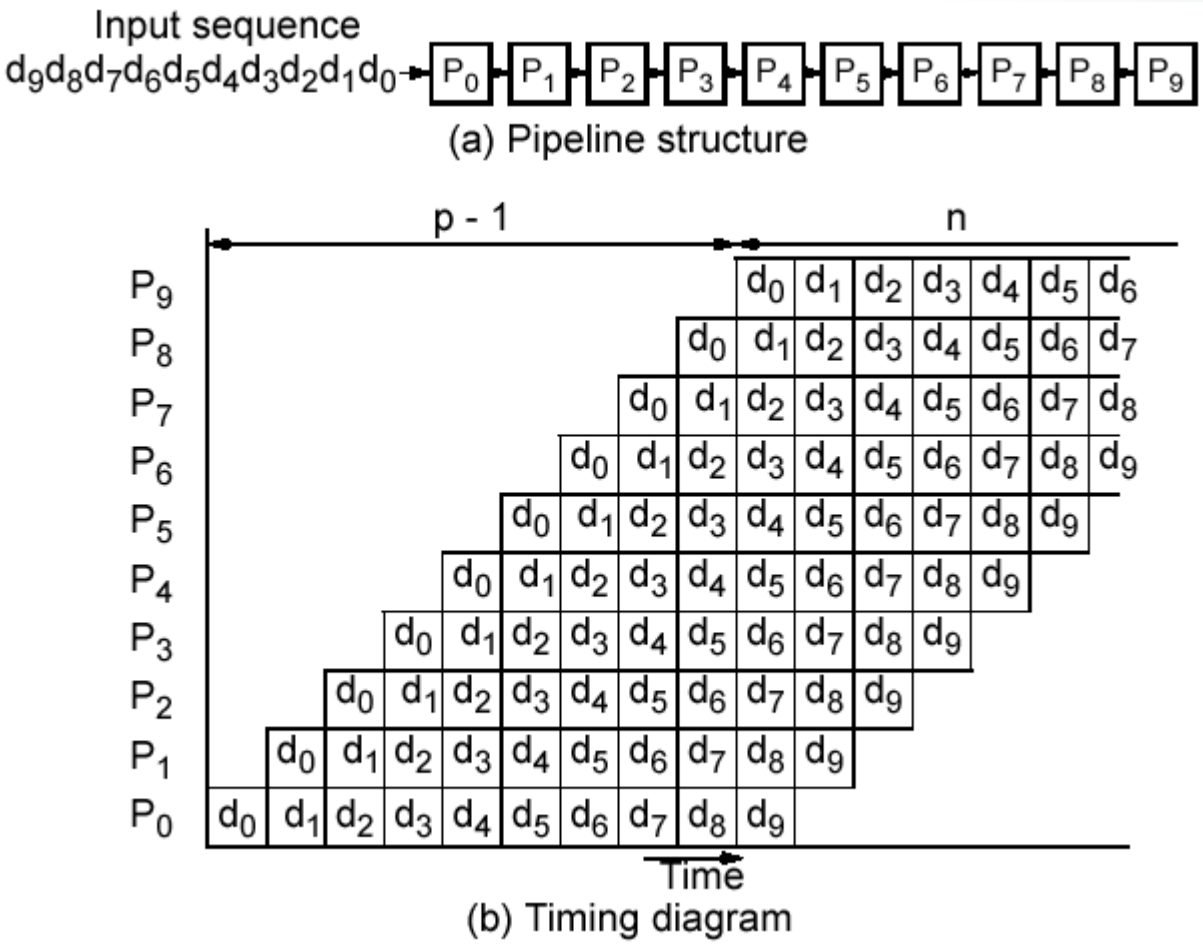
  - 将算法流程划分成p个前后衔接的任务片断，每个任务片断的输出作为下一个任务片断的输入；
  - 所有任务片断按同样的速率产生出结果。

❖ 流水线技术是一种广泛应用在并行处理中的技术；

并行算法设计方法

- 1. 流水线技术
- 下述2种类型的问题比较适合用流水线技术:
- （1）有多个实例需要执行
- 

（2）有一系列数据需要处理，并且每个数据需要多次进行操作



- 2. 分治策略：
- 设计思想：分而治之的原则是把一个问题分裂为若干个较小的子问题，这些问题可彼此独立地并行求解。
  - 数据分割：数据分割是指各结点基本执行相同的任务，只是数据不同。此方法常用在计算的各数据之间具有对称性的情况。例如对于向量的加法，可以分割为各个分量的加法。
  - 任务分割：任务分割是指总任务由自然的数个子任务组成，将其分摊在各个结点上。例如对于连续的数据处理任务，可以将其分割成数据采集、数据计算和结果输出三个子任务。
- 它是设计并行算法的根本准则 。

•例 计算$Y=(A+B(C+DEF))+G$

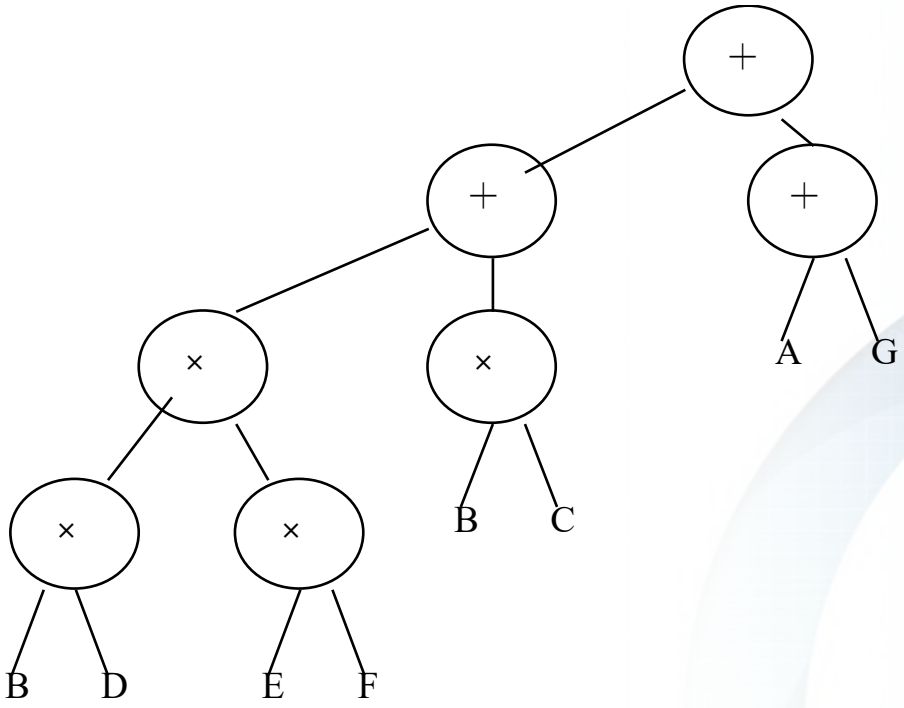
串行计算：需6步；

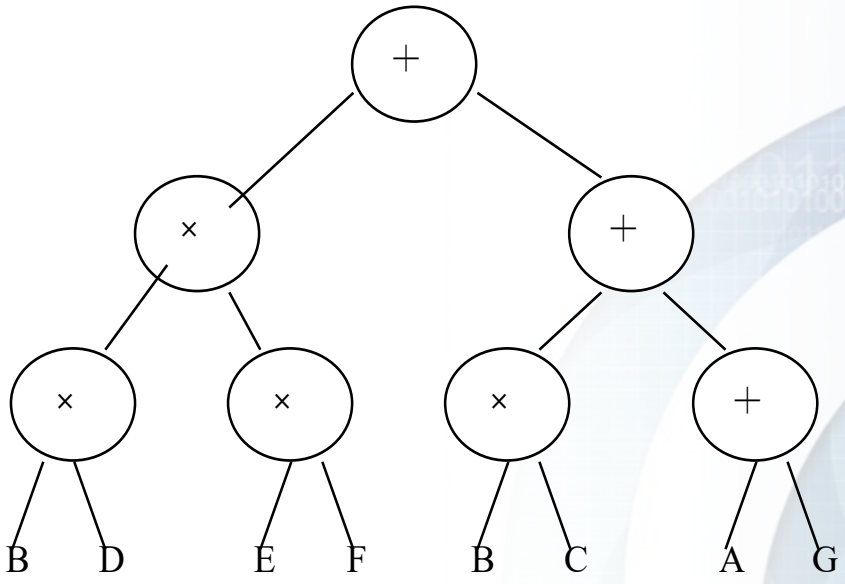
并行计算：

$$Y=Y_1+Y_2$$

$Y_1=A+G+BC$, $Y_2=BDEF$

如用两台计算机的系统，并行计算步为4；如用四台计算机的系统，并行计算步为3。


+
+
+
×
×
A
G
×
×
B
C
B
D
E
F


+
×
+
×
×
×
+
B
D
E
F
B
C
A
G

- 3.指针跳跃(pointer jumping)技术：
- 设计思想
  - 又称倍增设计技术，特别适合于处理链表或有向树之类的数据结构；
  - 每当递归调用时，所要处理的数据之间的距离将逐步加倍，经过步k后就可完成距离为2^k的所有数据的计算。
- 下面我们利用倍增技术来解决线性递归问题的并行计算。

$$\begin{cases} x_1 = b_1 \\ x_i = a_i x_{i-1} + b_i, i = 2,3,\ldots,n \end{cases}$$

$$Q(s,t)=\sum_{j=s}^{t}\left(\prod_{r=j+1}^{t}a_r\right)b_j$$

- 函数Q(s,t)有以下性质：
- （1）Q（i，i）＝b_i　i＝1，2，…，n；
- （2）当s<t，对于任意非负整数l，s＋l<t，我们有

$$Q(s,t)=\left(\prod_{r=s+l+1}^{t}a_r\right)Q(s,s+l)+Q(s+l+1,t)$$

❖ (3) Q (1, i) $=x_i$, i=1, 2, …, n。

$$\begin{cases} x_{2s} = (\prod_{r=s+1}^{2s} a_r) Q(1,s) + Q(s+1,2s) \\ x_{2s+1} = (\prod_{r=s+1}^{2s+1} a_r) Q(1,s) + Q(s+1,2s+1) \end{cases}$$

上述表达式即为一阶线性递归问题倍增法计算公式。

$$x_n = Q(1,2^m)$$

$$= (\prod_{r=2^{m-1}+1}^{2^m} a_r) Q(1,2^{m-1}) + Q(2^{m-1}+1,2^m)$$

例如取$n = 2^3, x_8 = Q(1,8)$ 的计算过程的递归分解情况如下:

第一次分解:

$$Q(1,8) = a_8 a_7 a_6 a_5 Q(1,4) + Q(5,8)$$

第二次分解:

$$Q(1,4) = a_4 a_3 Q(1,2) + Q(3,4)$$

$$Q(5,8) = a_8 a_7 Q(5,6) + Q(7,8)$$

❖ 第三次分解：

$$Q(1,2) = a_2 Q(1,1) + Q(2,2)$$
$$Q(3,4) = a_4 Q(3,3) + Q(4,4)$$
$$Q(5,6) = a_6 Q(5,5) + Q(6,6)$$
$$Q(7,8) = a_8 Q(7,7) + Q(8,8)$$

对于x7,x6,x5,x3也可以类似地进行3次分解，化为最简形式

❖ 一般情况，使用 n台处理机，计算n个元素值x_i $i = 1,2,\cdots,n$ ，需 $\lceil \log_2 n \rceil$ 步计算。

- 4.平衡树方法：
- 将输入元素作为叶节点构筑一棵平衡二叉树，然后自叶向根往返遍历。此法成功的部分原因是在树中能快速地存取所需要的信息。

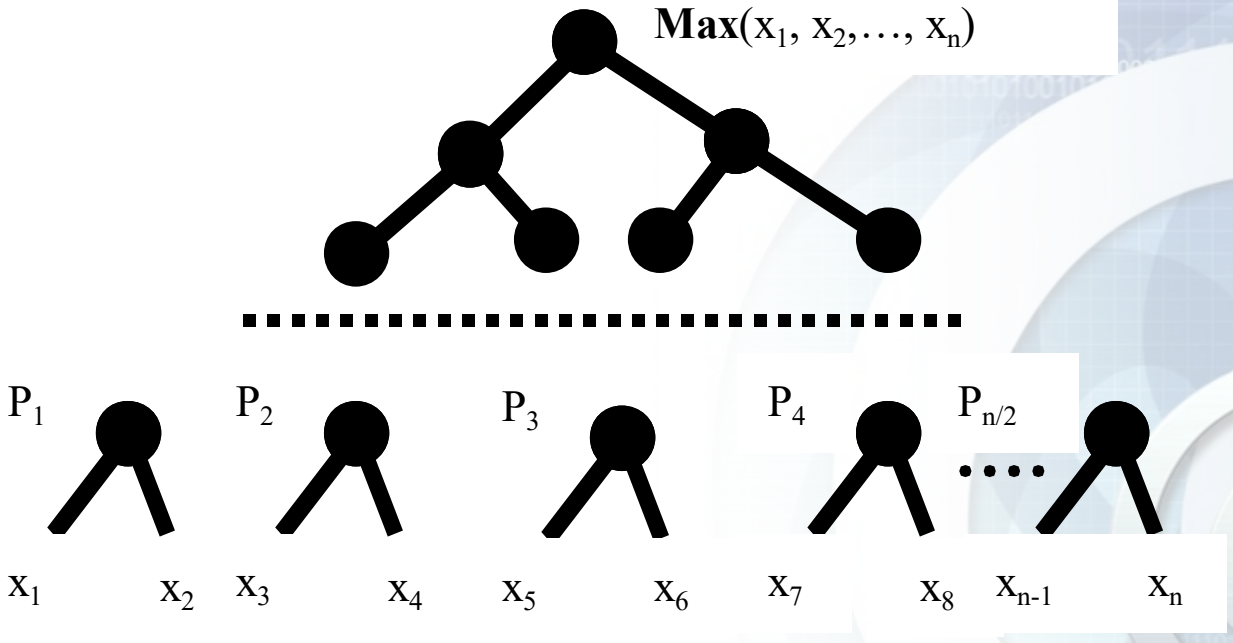


图 2－3　用平衡二叉树法求数的最大值

- 5. 加速级联策略：
- 为了获得一个快速最优的算法，我们可以将一个最优但相对不快的算法与另一个非常快但不是最优的算法级联（或串联）起来，形成一个加速级联算法 。
- 1）将输入划分成一些不相交的子输入；
- 2）开始使用最优算法，直到求解问题规模减到某一阈值；
- 3）接着使用快而非最优的算法，继续完成问题的求解。

- 当用双对数深度树求最大值时，每一层的最大值可在O（1）时间内计算完（在SIMD—CRCW模型上用枚举法[6]），因此，求最大值的时间T（n）＝O（loglogn），可见，它比平衡二叉树上求最大值要快指数倍。但总的运算量W（n）＝O（nloglogn）。
-

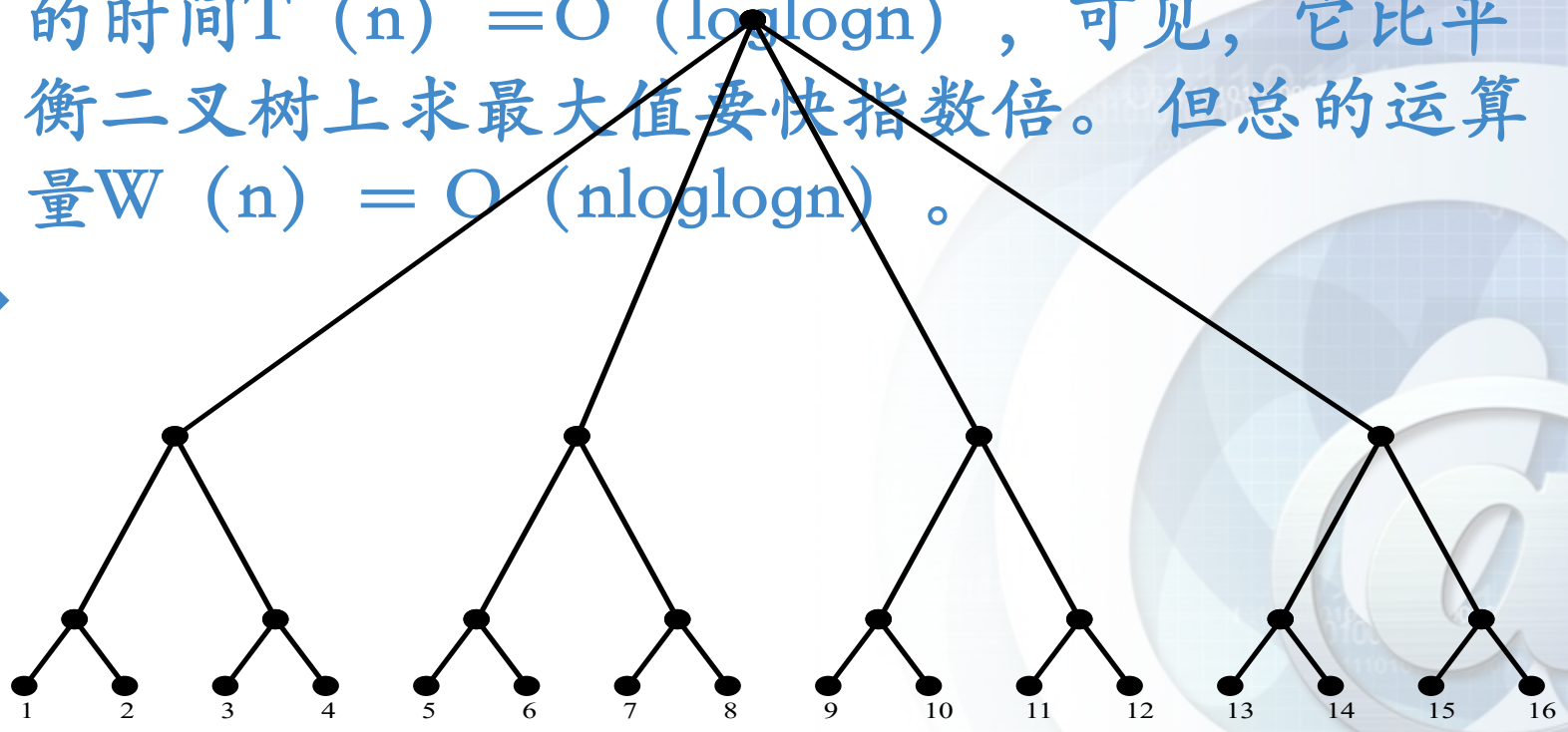


图 2—4　16 个结点的双对数深度树

- 第一步：运行对数深度的平衡二叉树算法，从叶结点向上直到层 $\lceil \log\log\log n \rceil$ 。 由于逐次向上每层将使候选的数目减少二分之一，所以在该算法结束时，最大数将处于 $n' = O(n/\log\log n)$ 个元素之中，并且至此所使用的运算量为$O(\log n)$，相应的时间为$O(\log\log n)$ 。

- 第二步：对第一步所产生的 n' 个最大值的候选数，运行双对数深度树算法。此步所使用的运算量 $w(n') = O(n'\log\log n') = O(n)$，相应的时间为 $O(\log\log n)$。

- 因此，整个算法的运行时间为 $O(\log\log n)$ 而运算量为 $O(n)$。

并行算法的复杂性度量

时间复杂度	运算次数
空间复杂度	存储占用，处理机台数
问题的规模	问题输入数据的度量

时间复杂度

矩阵向量乘法伪代码：

```
for (int row=0;row< nRows; row++){
        T temp= (T) 0;
         for (int col = 0; col < nCols; col++){
                temp + = matrix [row*nCols +col]*vector[col];
       }
          product[row]+= temp;
}
```

- 总计算量 nRows*(4*nCols+2)，若是方阵，nRows=nCols=n，总计算量为4*n^2+2n，时间复杂度 $O(n^2)$

空间复杂度

- 如前矩阵向量乘积算法，输入矩阵占用空间大小为 n^2，输入向量和输出向量大小均是**n**，其使用的空间大小为 $n^2 + 2 \times n$，其空间复杂度为 $O(n^2)$.

并行算法的复杂性度量

- 并行算法的复杂性度量
  - 最坏情况下的复杂度(Worst-CASE Complexity)
  - 期望复杂度(Expected Complexity)
- 并行算法的几个复杂性度量指标
  - **运行时间t(n):**包含计算时间和通讯时间，分别用计算时间步和选路时间步作单位。n为问题实例的输入规模。
  - **处理器数p(n)**
  - **并行算法成本c(n):** $c(n)=t(n)p(n)$
  - **总运算量W(n):** 并行算法求解问题时所完成的总的操作步数。

并行算法的复杂性度量

❖ Brent定理

令W(n)是某并行算法A在运行时间T(n)内所执行的运算量，则A使用*p*台处理器可在t(n)=O(W(n)/p+T(n))时间内执行完毕。

- W(n)和c(n)密切相关
- P=O(W(n)/T(n))时，W(n)和c(n)两者是渐进一致的
- 对于任意的p，c(n)›W(n)

❖证明

证明 令 $W_i(n)$ 是在时间 $i(1 \leqslant i \leqslant T(n))$ 所执行的运算数量，则使用 p 台处理器可在小于等于 $\lceil W_i(n)/p \rceil$ 并行步内模拟它。如果模拟是成功的，则在相应于 p 台处理器的 SIMD 共享存储模型上共花费了小于等于 $\sum_i \lceil W_i(n)/p \rceil$ 并行步。而 $\sum_i \lceil W_i(n)/p \rceil \leqslant \sum_i (\lfloor W_i(n)/p \rfloor + 1) \leqslant W(n)/p + T(n)$。所以定理得证。□

使用 Brent 定理的关键是如何尽可能地将每个时间步的计算量 $W_i(n)$ 均摊给 p 台处理器；而且每个处理器均必须知道它是否是活动的，如果是活动的，它还必须知道它所应执行的指令和相应的操作数等。

时间复杂度

- 定义：如果一个问题的大小为n，解决此问题的某一算法所需的时间为T(n)，则称T(n)为算法的时间复杂度
- 一般而言，我们关心的是n充分大时的复杂度，此时它与渐近复杂度相差不多，经常使用上界、下界、紧致界的概念

时间复杂度

- 上界：

  令**f(n)**和**g(n)**是定义在自然数集合**N**上的两个函数，如存在两个正常数**c**和n_0，使得对于所有$n \geq n_0$均有$f(n) \leq cg(n)$，则称**g(n)**是**f(n)**的一个上界，记作$f(n)=O(g(n))$

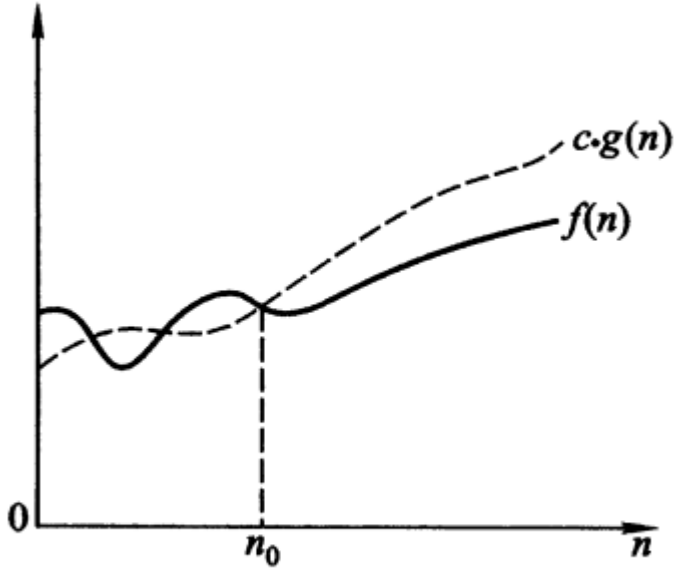
- 例：
  - $4n=O(n)$
  - $5n^2=O(n^2)$

时间复杂度

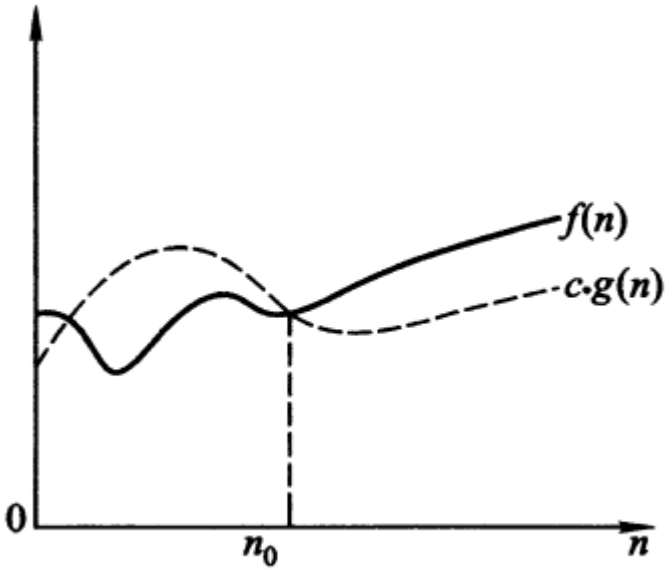
- 下界：

令**f(n)**和**g(n)**是定义在自然数集合**N**上的两个函数，如存在两个正常数**c**和n_0，使得对于所有$n \geq n_0$均有$f(n) \geq cg(n)$，则称**g(n)**是**f(n)**的一个下界，记作$f(n)=\Omega(g(n))$

上界、下界



(a) 上界



(b) 下界

时间复杂度

- 紧致界：

令f(n)和g(n)是定义在自然数集合N上的两个函数，如存在两个正常数c1、c2和n_0，使得对于所有$n \geq n_0$均有$c1*g(n) \leq f(n) \leq c2*g(n)$，则称g(n)是f(n)的一个精确界，又称紧致界，记作

$f(n)=\Theta(g(n))$

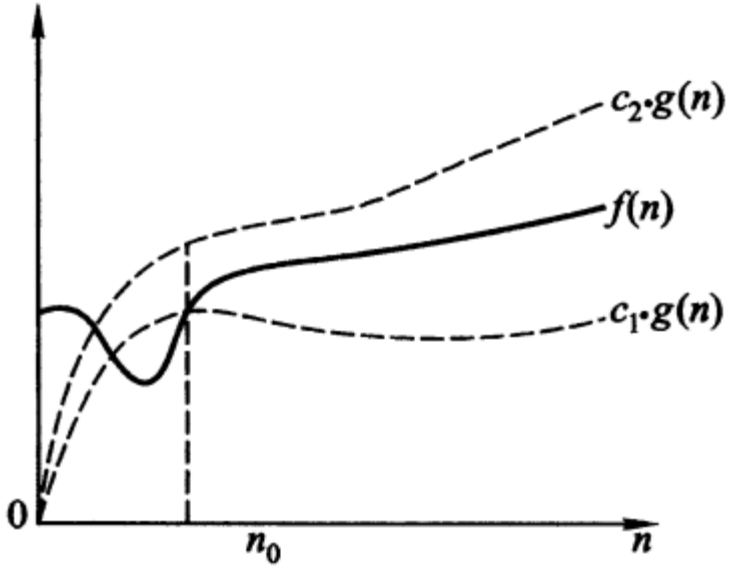
时间复杂度

- 紧致界:

  令f(n)和g(n)是定义在自然数集合N上的两个函数，如存在两个正常数c_1，c_2和n_0，使得对于所有$n \geq n_0$均有$c_1 g(n) \leq f(n) \leq c_2 g(n)$，则称g(n)是f(n)的一个紧致界，记作$f(n)=\theta(g(n))$

- 我们衡量算法的复杂度用上界，即最坏情况下的复杂度。

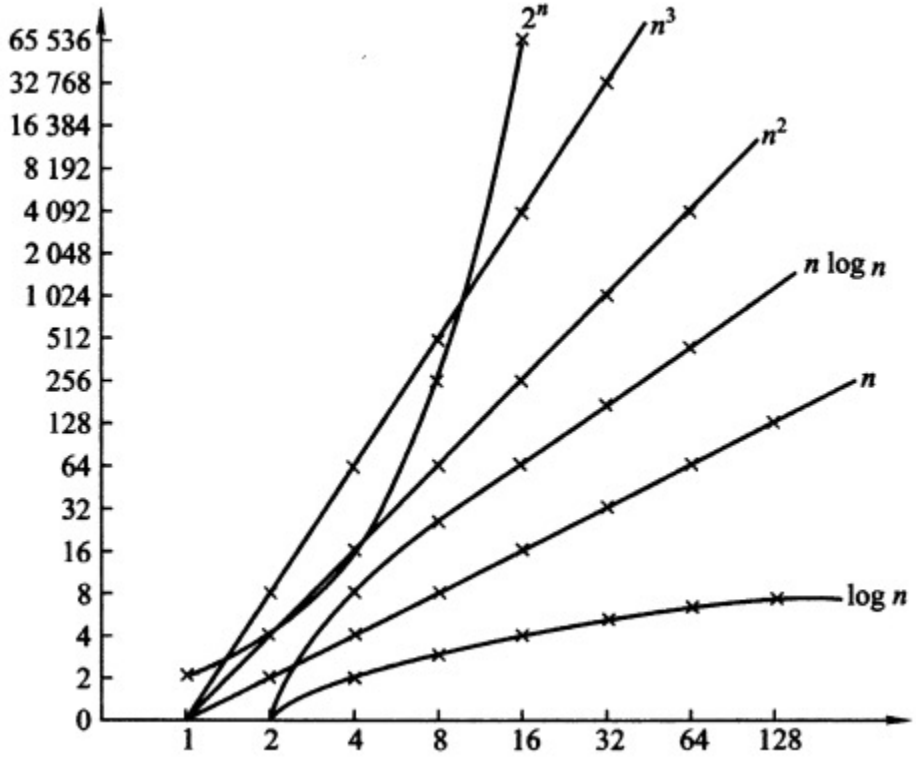
紧致界的几何解释



（c）紧致界

几种常见的函数曲线

- $O(1)<O(\log n)<O(n)<O(n\log n)<O(n^2)<O(2^n)<O(n!)<O(n^n)$



计算复杂度对计算时间的影响

- 当代计算机速度以惊人的变化增长，人们误认为算法的好坏对计算时间的影响不会太大，计算复杂性不重要，但事实不是如此。
- 例：求解一个规模为n的问题

  算法A1:O(n)　　A2:O(2^n)

  问题的计算规模n=60时，用A1需要0.0006s

  用A2需要366世纪。

并行算法性能度量

- 1. 运行时间：算法在并行计算机上求解一个问题所需要的时间，即算法开始执行到执行结束的这一段时间。

- 2. 并行度：该算法中可并行执行的操作数。

  并行度与任务粒度的关系：一般而言，大的粒度意味着能独立并行执行的是大任务，算法的并行度小；小的粒度意味着能独立并行执行的是小任务，算法的并行度大。

- 3. 成本：并行算法的运行时间T(n)与并行算法所需要的处理机数P(n)的乘积，即

$$C(n)=T(n)*P(n)$$

- 4. 加速比和效率：

$$S_P = \frac{T_1}{T_P}$$

- T_1指串行算法在单处理机上的运行时间；T_P指并行算法在并行处理机使用P台处理机所需时间；

$$E_P = \frac{S_P}{P}$$

❖ 说明：

(1) $S_p(n) \le p$ 线性加速比；

(2) $S_p(n) > p$ 超线性加速比；

一般情况下是不会发生的（额外开销），但某些情形下会发生。

(3) 实用的加速比定义：

$$S_P = \frac{\text{串行算法的运行时间}}{\text{并行算法的运行时间}}$$

- 5.并行算法的可扩展性分析：

并行算法的可扩展性是对并行算法有效利用增加的处理机数目能力的一种度量，与算法设计内在并行性和通信需求有关。

- 对某些并行算法而言，在处理器数增加的情况下，只要适当地增加问题规模，可以保持算法的效率不变，此即并行算法可扩展性的恒等效率度量方法。
- 假设某并行算法在P台同构处理及组成的并行计算机上执行时间为T_P，他对应的最优串行算法的执行时间为T_1，则根据并行算法的效率定义为：

$$E_P = \frac{S_P}{P} = \frac{T_1}{T_P \times P} = \frac{T_1}{T_1 + T_o} = \frac{1}{1 + T_o / T_1}$$

四并行加速比模型

- 1. Amdahl加速比模型：

$$S_P = \frac{f + N}{f + N/P} = \frac{1}{f + N/P}$$

- 2. Gustafson加速比模型：

  1988年2月美国Sandia国家实验室的Gustafson等三人在含1024个处理器的nCUBE系统上进行了非稳定流修正、曲面波模拟以及薄膜应力分析等3个应用问题的试算，实际加速比分别达到了1016、1020与1021！

$$S_P = \frac{f + N \cdot P}{f + (N \cdot P)/P} = f + N \cdot P$$